

**Course Title : HIGH PERFORMANCE COMPUTING Lab Ass-5**

**Name-G Datha Narayana**

**roll no:-2303A51754**

**Batch-11**

Lab: Loop Scheduling Comparison

Task 1: Code (reuse Week-3 code, only change directive)

```
#pragma omp parallel for schedule(runtime)
```

```
reduction(+:sum)
```

```
for (long i = 0; i < N; i++) {
```

```
sum += i;
```

```
}
```

Run commands:

```
gcc -fopenmp sum_openmp.c -o sum_openmp
```

```
export OMP_SCHEDULE=static
```

```
export OMP_NUM_THREADS=8
```

```
./sum_openmp
```

```
export OMP_SCHEDULE=dynamic
```

04-02-2026 EOD

```
./sum_openmp
```

```
export OMP_SCHEDULE=guided
```

```
./sum_openmp
```

Fill the table below:

Scheduling Time Observation

Static ? ?

Dynamic ? ?

Guided ? ?

```

GNU nano 7.2                                sum_serial.c
## Task 1: Loop Scheduling Comparison (schedule(runtime))

```c
#include <stdio.h>
#include <omp.h>

int main() {
    long N = 100000000;
    long long sum = 0;

    double start = omp_get_wtime();

    #pragma omp parallel for schedule(runtime) reduction(+:sum)
    for (long i = 0; i < N; i++) {
        sum += i;
    }

    double end = omp_get_wtime();

    printf("Sum = %lld\n", sum);
    printf("Time taken = %f seconds\n", end - start);

    return 0;
}
```

[ Read 106 lines ]
AG Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark
AX Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo     M-G Copy

nithish@LAPTOP-A2F0J865:/mnt/c/Users/Sreeshma$ nano sum_openmp.c
nithish@LAPTOP-A2F0J865:/mnt/c/Users/Sreeshma$ gcc -fopenmp sum_openmp.c -o sum_openmp
nithish@LAPTOP-A2F0J865:/mnt/c/Users/Sreeshma$ export OMP_NUM_THREADS=8

export OMP_SCHEDULE=static
./sum_openmp

export OMP_SCHEDULE=dynamic
./sum_openmp

export OMP_SCHEDULE=guided
./sum_openmp
Sum = 4999999950000000
Time taken = 0.115592 seconds
Sum = 4999999950000000
Time taken = 4.541940 seconds
Sum = 4999999950000000
Time taken = 0.055931 seconds

```

| Scheduling | Time             | Observation      |
|------------|------------------|------------------|
| Static     | 0.115592 seconds | 4999999950000000 |
| Dynamic    | 4.541940 seconds | 4999999950000000 |
| Guided     | 0.055931 seconds | 4999999950000000 |

2

Task 2: Synchronization Overhead Comparison

Variant-1: Reduction

#pragma omp parallel for reduction(+:sum)

Variant-2: Atomic

#pragma omp parallel for

for (...) {

```
#pragma omp atomic  
sum += i;  
}
```

Variant-3: Critical

```
#pragma omp parallel for
```

04-02-2026 EOD

```
for (...) {  
#pragma omp critical  
sum += i;  
}
```

Expected trend:

Reduction → fastest

Atomic → slower

Critical → slowest

## Reduction:

```
GNU nano 7.2 reduction.c
#include <stdio.h>
#include <omp.h>

int main() {
    long N = 1000000000;
    long long sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < N; i++) {
        sum += i;
    }

    printf("Reduction Sum = %lld\n", sum);
    return 0;
}

[ Read 15 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ gcc -fopenmp reduction.c -o r
eduction
./reduction
Reduction Sum = 499999999500000000
```

## Atomic:

```

GNU nano 7.2                                atomic.c *
#include <stdio.h>
#include <omp.h>

int main() {
    long N = 1000000000;
    long long sum = 0;

    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        #pragma omp atomic
        sum += i;
    }

    printf("Atomic Sum = %lld\n", sum);
    return 0;
}

```

```

nithish@LAPTOP-A2F0J865:/mnt/c/Users/Sreeshma$ nano atomic.c
nithish@LAPTOP-A2F0J865:/mnt/c/Users/Sreeshma$ gcc -fopenmp atomic.c -o atom
ic
./atomic

Atomic Sum = 499999999500000000

```

**Critical:**

```
GNU nano 7.2                                critical.c *
#include <stdio.h>
#include <omp.h>

int main() {
    long N = 1000000000;
    long long sum = 0;

    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        #pragma omp critical
        sum += i;
    }

    printf("Critical Sum = %lld\n", sum);
    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ gcc -fopenmp critical.c -o critical
./critical
Critical Sum = 499999999500000000
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$
```